Rerunning Flaky Tests under Controlled Variations

Anonymous Author(s)
Anonymous Institution

***Abstract*—Flaky tests pass and fail on the same code. This paper reruns each failing test under controlled variations.**

I. Introduction

Continuous integration runs the test suite of a project after every change to the code. A failing test stops the merge of the change until a developer finds the cause of the failure and repairs the code or the test. Most failures point to a fault in the change itself. Some failures, however, come from tests that pass and fail on the same code, and these flaky tests cost developers hours of work on faults that do not exist.

Prior studies of open-source projects report that flaky tests account for a large part of the failures that developers investigate. The most common causes are asynchronous waits, concurrency, the order in which tests run, and dependencies on the network or on the clock. Each cause calls for a different repair. A test that waits for a fixed time needs a condition to wait for, while a test that depends on the order of the suite needs its own setup and teardown of the shared state.

The detector of this paper reruns each failing test under controlled variations of its environment. It changes the order of the tests, the timing of the threads, and the seed of every random number generator. A test whose outcome changes under any variation is flaky, and the variation that changes the outcome names the likely cause. The detector reports the test, the variation, and the lines of the test that read the shared state.

We evaluate the detector on the failures of twelve Java projects over one year of builds. The projects range from small libraries with a few hundred tests to large frameworks with tens of thousands of tests. For each failure, the build logs record the commit, the failing tests, and the outcome of the next build on the same commit. A failure that the next build does not repeat is a candidate flaky failure, which two authors then label by hand.

The detector finds the cause of most flaky failures within ten reruns. Order dependence and timing account for the largest groups, followed by network access and random seeds. The remaining failures depend on the state of the build machine, such as a full disk or a busy port, which no variation of the test environment reproduces. These failures call for checks of the machine before the build instead of changes to the test.

Developers accepted most of the repairs that the detector suggested. A repair that adds a condition to a wait or a reset of a shared field takes a few lines of code and leaves the intent of the test intact. Repairs of tests that depend on the network were less popular, since they replace a real service with a stub and thus test less of the system. Developers preferred to keep such tests and to mark them for a separate suite that runs once a day.

The cost of the detector lies in the reruns. Ten reruns of a failing test take on average a few seconds for a unit test and a few minutes for an integration test. Since flaky failures are a small part of all builds, the reruns add little to the total time of the pipeline. The detector also caches the outcome of each variation, therefore a test that fails again in a later build needs no new reruns until its code changes.

Several threats limit these results. The labels of the failures come from two authors, who agreed on most cases and resolved the rest in a discussion. The projects use Java and Maven, and other languages and build tools may show other causes of flakiness. Finally, one year of builds may miss causes that appear only after a change of the build machines, such as a new version of the operating system or of the virtual machine. Continuous integration runs the test suite of a project after every change to the code. A failing test stops the merge of the change until a developer finds the cause of the failure and repairs the code or the test. Most failures point to a fault in the change itself. Some failures, however, come from tests that pass and fail on the same code, and these flaky tests cost developers hours of work on faults that do not exist.

Prior studies of open-source projects report that flaky tests account for a large part of the failures that developers investigate. The most common causes are asynchronous waits, concurrency, the order in which tests run, and dependencies on the network or on the clock. Each cause calls for a different repair. A test that waits for a fixed time needs a condition to wait for, while a test that depends on the order of the suite needs its own setup and teardown of the shared state.

The detector of this paper reruns each failing test under controlled variations of its environment. It changes the order of the tests, the timing of the threads, and the seed of every random number generator. A test whose outcome changes under any variation is flaky, and the variation that changes the outcome names the likely cause. The detector reports the test, the variation, and the lines of the test that read the shared state.

We evaluate the detector on the failures of twelve Java projects over one year of builds. The projects range from small libraries with a few hundred tests to large frameworks with tens of thousands of tests. For each failure, the build logs record the commit, the failing tests, and the outcome of the next build on the same commit. A failure that the next build does not repeat is a candidate flaky failure, which two authors then label by hand.

The detector finds the cause of most flaky failures within ten

reruns. Order dependence and timing account for the largest groups, followed by network access and random seeds. The remaining failures depend on the state of the build machine, such as a full disk or a busy port, which no variation of the test environment reproduces. These failures call for checks of the machine before the build instead of changes to the test.

Developers accepted most of the repairs that the detector suggested. A repair that adds a condition to a wait or a reset of a shared field takes a few lines of code and leaves the intent of the test intact. Repairs of tests that depend on the network were less popular, since they replace a real service with a stub and thus test less of the system. Developers preferred to keep such tests and to mark them for a separate suite that runs once a day.

The cost of the detector lies in the reruns. Ten reruns of a failing test take on average a few seconds for a unit test and a few minutes for an integration test. Since flaky failures are a small part of all builds, the reruns add little to the total time of the pipeline. The detector also caches the outcome of each variation, therefore a test that fails again in a later build needs no new reruns until its code changes.

Several threats limit these results. The labels of the failures come from two authors, who agreed on most cases and resolved the rest in a discussion. The projects use Java and Maven, and other languages and build tools may show other causes of flakiness. Finally, one year of builds may miss causes that appear only after a change of the build machines, such as a new version of the operating system or of the virtual machine.

II. Approach

Continuous integration runs the test suite of a project after every change to the code. A failing test stops the merge of the change until a developer finds the cause of the failure and repairs the code or the test. Most failures point to a fault in the change itself. Some failures, however, come from tests that pass and fail on the same code, and these flaky tests cost developers hours of work on faults that do not exist.

Prior studies of open-source projects report that flaky tests account for a large part of the failures that developers investigate. The most common causes are asynchronous waits, concurrency, the order in which tests run, and dependencies on the network or on the clock. Each cause calls for a different repair. A test that waits for a fixed time needs a condition to wait for, while a test that depends on the order of the suite needs its own setup and teardown of the shared state.

The detector of this paper reruns each failing test under controlled variations of its environment. It changes the order of the tests, the timing of the threads, and the seed of every random number generator. A test whose outcome changes under any variation is flaky, and the variation that changes the outcome names the likely cause. The detector reports the test, the variation, and the lines of the test that read the shared state.

We evaluate the detector on the failures of twelve Java projects over one year of builds. The projects range from small libraries with a few hundred tests to large frameworks with tens of thousands of tests. For each failure, the build logs record the commit, the failing tests, and the outcome of the next build on the same commit. A failure that the next build does not repeat is a candidate flaky failure, which two authors then label by hand.

The detector finds the cause of most flaky failures within ten reruns. Order dependence and timing account for the largest groups, followed by network access and random seeds. The remaining failures depend on the state of the build machine, such as a full disk or a busy port, which no variation of the test environment reproduces. These failures call for checks of the machine before the build instead of changes to the test.

Developers accepted most of the repairs that the detector suggested. A repair that adds a condition to a wait or a reset of a shared field takes a few lines of code and leaves the intent of the test intact. Repairs of tests that depend on the network were less popular, since they replace a real service with a stub and thus test less of the system. Developers preferred to keep such tests and to mark them for a separate suite that runs once a day.

The cost of the detector lies in the reruns. Ten reruns of a failing test take on average a few seconds for a unit test and a few minutes for an integration test. Since flaky failures are a small part of all builds, the reruns add little to the total time of the pipeline. The detector also caches the outcome of each variation, therefore a test that fails again in a later build needs no new reruns until its code changes.

Several threats limit these results. The labels of the failures come from two authors, who agreed on most cases and resolved the rest in a discussion. The projects use Java and Maven, and other languages and build tools may show other causes of flakiness. Finally, one year of builds may miss causes that appear only after a change of the build machines, such as a new version of the operating system or of the virtual machine. Continuous integration runs the test suite of a project after every change to the code. A failing test stops the merge of the change until a developer finds the cause of the failure and repairs the code or the test. Most failures point to a fault in the change itself. Some failures, however, come from tests that pass and fail on the same code, and these flaky tests cost developers hours of work on faults that do not exist.

Prior studies of open-source projects report that flaky tests account for a large part of the failures that developers investigate. The most common causes are asynchronous waits, concurrency, the order in which tests run, and dependencies on the network or on the clock. Each cause calls for a different repair. A test that waits for a fixed time needs a condition to wait for, while a test that depends on the order of the suite needs its own setup and teardown of the shared state.

The detector of this paper reruns each failing test under controlled variations of its environment. It changes the order of the tests, the timing of the threads, and the seed of every random number generator. A test whose outcome changes under any variation is flaky, and the variation that changes the outcome names the likely cause. The detector reports the

test, the variation, and the lines of the test that read the shared state.

We evaluate the detector on the failures of twelve Java projects over one year of builds. The projects range from small libraries with a few hundred tests to large frameworks with tens of thousands of tests. For each failure, the build logs record the commit, the failing tests, and the outcome of the next build on the same commit. A failure that the next build does not repeat is a candidate flaky failure, which two authors then label by hand.

The detector finds the cause of most flaky failures within ten reruns. Order dependence and timing account for the largest groups, followed by network access and random seeds. The remaining failures depend on the state of the build machine, such as a full disk or a busy port, which no variation of the test environment reproduces. These failures call for checks of the machine before the build instead of changes to the test.

Developers accepted most of the repairs that the detector suggested. A repair that adds a condition to a wait or a reset of a shared field takes a few lines of code and leaves the intent of the test intact. Repairs of tests that depend on the network were less popular, since they replace a real service with a stub and thus test less of the system. Developers preferred to keep such tests and to mark them for a separate suite that runs once a day.

The cost of the detector lies in the reruns. Ten reruns of a failing test take on average a few seconds for a unit test and a few minutes for an integration test. Since flaky failures are a small part of all builds, the reruns add little to the total time of the pipeline. The detector also caches the outcome of each variation, therefore a test that fails again in a later build needs no new reruns until its code changes.

Several threats limit these results. The labels of the failures come from two authors, who agreed on most cases and resolved the rest in a discussion. The projects use Java and Maven, and other languages and build tools may show other causes of flakiness. Finally, one year of builds may miss causes that appear only after a change of the build machines, such as a new version of the operating system or of the virtual machine.

III. Evaluation

Continuous integration runs the test suite of a project after every change to the code. A failing test stops the merge of the change until a developer finds the cause of the failure and repairs the code or the test. Most failures point to a fault in the change itself. Some failures, however, come from tests that pass and fail on the same code, and these flaky tests cost developers hours of work on faults that do not exist.

Prior studies of open-source projects report that flaky tests account for a large part of the failures that developers investigate. The most common causes are asynchronous waits, concurrency, the order in which tests run, and dependencies on the network or on the clock. Each cause calls for a different repair. A test that waits for a fixed time needs a condition to wait for, while a test that depends on the order of the suite needs its own setup and teardown of the shared state.

The detector of this paper reruns each failing test under controlled variations of its environment. It changes the order of the tests, the timing of the threads, and the seed of every random number generator. A test whose outcome changes under any variation is flaky, and the variation that changes the outcome names the likely cause. The detector reports the test, the variation, and the lines of the test that read the shared state.

We evaluate the detector on the failures of twelve Java projects over one year of builds. The projects range from small libraries with a few hundred tests to large frameworks with tens of thousands of tests. For each failure, the build logs record the commit, the failing tests, and the outcome of the next build on the same commit. A failure that the next build does not repeat is a candidate flaky failure, which two authors then label by hand.

The detector finds the cause of most flaky failures within ten reruns. Order dependence and timing account for the largest groups, followed by network access and random seeds. The remaining failures depend on the state of the build machine, such as a full disk or a busy port, which no variation of the test environment reproduces. These failures call for checks of the machine before the build instead of changes to the test.

Developers accepted most of the repairs that the detector suggested. A repair that adds a condition to a wait or a reset of a shared field takes a few lines of code and leaves the intent of the test intact. Repairs of tests that depend on the network were less popular, since they replace a real service with a stub and thus test less of the system. Developers preferred to keep such tests and to mark them for a separate suite that runs once a day.

The cost of the detector lies in the reruns. Ten reruns of a failing test take on average a few seconds for a unit test and a few minutes for an integration test. Since flaky failures are a small part of all builds, the reruns add little to the total time of the pipeline. The detector also caches the outcome of each variation, therefore a test that fails again in a later build needs no new reruns until its code changes.

Several threats limit these results. The labels of the failures come from two authors, who agreed on most cases and resolved the rest in a discussion. The projects use Java and Maven, and other languages and build tools may show other causes of flakiness. Finally, one year of builds may miss causes that appear only after a change of the build machines, such as a new version of the operating system or of the virtual machine. Continuous integration runs the test suite of a project after every change to the code. A failing test stops the merge of the change until a developer finds the cause of the failure and repairs the code or the test. Most failures point to a fault in the change itself. Some failures, however, come from tests that pass and fail on the same code, and these flaky tests cost developers hours of work on faults that do not exist.

Prior studies of open-source projects report that flaky tests account for a large part of the failures that developers in-

vestigate. The most common causes are asynchronous waits, concurrency, the order in which tests run, and dependencies on the network or on the clock. Each cause calls for a different repair. A test that waits for a fixed time needs a condition to wait for, while a test that depends on the order of the suite needs its own setup and teardown of the shared state.

The detector of this paper reruns each failing test under controlled variations of its environment. It changes the order of the tests, the timing of the threads, and the seed of every random number generator. A test whose outcome changes under any variation is flaky, and the variation that changes the outcome names the likely cause. The detector reports the test, the variation, and the lines of the test that read the shared state.

We evaluate the detector on the failures of twelve Java projects over one year of builds. The projects range from small libraries with a few hundred tests to large frameworks with tens of thousands of tests. For each failure, the build logs record the commit, the failing tests, and the outcome of the next build on the same commit. A failure that the next build does not repeat is a candidate flaky failure, which two authors then label by hand.

The detector finds the cause of most flaky failures within ten reruns. Order dependence and timing account for the largest groups, followed by network access and random seeds. The remaining failures depend on the state of the build machine, such as a full disk or a busy port, which no variation of the test environment reproduces. These failures call for checks of the machine before the build instead of changes to the test.

Developers accepted most of the repairs that the detector suggested. A repair that adds a condition to a wait or a reset of a shared field takes a few lines of code and leaves the intent of the test intact. Repairs of tests that depend on the network were less popular, since they replace a real service with a stub and thus test less of the system. Developers preferred to keep such tests and to mark them for a separate suite that runs once a day.

The cost of the detector lies in the reruns. Ten reruns of a failing test take on average a few seconds for a unit test and a few minutes for an integration test. Since flaky failures are a small part of all builds, the reruns add little to the total time of the pipeline. The detector also caches the outcome of each variation, therefore a test that fails again in a later build needs no new reruns until its code changes.

Several threats limit these results. The labels of the failures come from two authors, who agreed on most cases and resolved the rest in a discussion. The projects use Java and Maven, and other languages and build tools may show other causes of flakiness. Finally, one year of builds may miss causes that appear only after a change of the build machines, such as a new version of the operating system or of the virtual machine.

IV. Discussion

Continuous integration runs the test suite of a project after every change to the code. A failing test stops the merge of the change until a developer finds the cause of the failure and repairs the code or the test. Most failures point to a fault in the change itself. Some failures, however, come from tests that pass and fail on the same code, and these flaky tests cost developers hours of work on faults that do not exist.

Prior studies of open-source projects report that flaky tests account for a large part of the failures that developers investigate. The most common causes are asynchronous waits, concurrency, the order in which tests run, and dependencies on the network or on the clock. Each cause calls for a different repair. A test that waits for a fixed time needs a condition to wait for, while a test that depends on the order of the suite needs its own setup and teardown of the shared state.

The detector of this paper reruns each failing test under controlled variations of its environment. It changes the order of the tests, the timing of the threads, and the seed of every random number generator. A test whose outcome changes under any variation is flaky, and the variation that changes the outcome names the likely cause. The detector reports the test, the variation, and the lines of the test that read the shared state.

We evaluate the detector on the failures of twelve Java projects over one year of builds. The projects range from small libraries with a few hundred tests to large frameworks with tens of thousands of tests. For each failure, the build logs record the commit, the failing tests, and the outcome of the next build on the same commit. A failure that the next build does not repeat is a candidate flaky failure, which two authors then label by hand.

The detector finds the cause of most flaky failures within ten reruns. Order dependence and timing account for the largest groups, followed by network access and random seeds. The remaining failures depend on the state of the build machine, such as a full disk or a busy port, which no variation of the test environment reproduces. These failures call for checks of the machine before the build instead of changes to the test.

Developers accepted most of the repairs that the detector suggested. A repair that adds a condition to a wait or a reset of a shared field takes a few lines of code and leaves the intent of the test intact. Repairs of tests that depend on the network were less popular, since they replace a real service with a stub and thus test less of the system. Developers preferred to keep such tests and to mark them for a separate suite that runs once a day.

The cost of the detector lies in the reruns. Ten reruns of a failing test take on average a few seconds for a unit test and a few minutes for an integration test. Since flaky failures are a small part of all builds, the reruns add little to the total time of the pipeline. The detector also caches the outcome of each variation, therefore a test that fails again in a later build needs no new reruns until its code changes.

Several threats limit these results. The labels of the failures come from two authors, who agreed on most cases and resolved the rest in a discussion. The projects use Java and Maven, and other languages and build tools may show other causes of flakiness. Finally, one year of builds may miss causes

that appear only after a change of the build machines, such as a new version of the operating system or of the virtual machine. Continuous integration runs the test suite of a project after every change to the code. A failing test stops the merge of the change until a developer finds the cause of the failure and repairs the code or the test. Most failures point to a fault in the change itself. Some failures, however, come from tests that pass and fail on the same code, and these flaky tests cost developers hours of work on faults that do not exist.

Prior studies of open-source projects report that flaky tests account for a large part of the failures that developers investigate. The most common causes are asynchronous waits, concurrency, the order in which tests run, and dependencies on the network or on the clock. Each cause calls for a different repair. A test that waits for a fixed time needs a condition to wait for, while a test that depends on the order of the suite needs its own setup and teardown of the shared state.

The detector of this paper reruns each failing test under controlled variations of its environment. It changes the order of the tests, the timing of the threads, and the seed of every random number generator. A test whose outcome changes under any variation is flaky, and the variation that changes the outcome names the likely cause. The detector reports the test, the variation, and the lines of the test that read the shared state.

We evaluate the detector on the failures of twelve Java projects over one year of builds. The projects range from small libraries with a few hundred tests to large frameworks with tens of thousands of tests. For each failure, the build logs record the commit, the failing tests, and the outcome of the next build on the same commit. A failure that the next build does not repeat is a candidate flaky failure, which two authors then label by hand.

The detector finds the cause of most flaky failures within ten reruns. Order dependence and timing account for the largest groups, followed by network access and random seeds. The remaining failures depend on the state of the build machine, such as a full disk or a busy port, which no variation of the test environment reproduces. These failures call for checks of the machine before the build instead of changes to the test.

Developers accepted most of the repairs that the detector suggested. A repair that adds a condition to a wait or a reset of a shared field takes a few lines of code and leaves the intent of the test intact. Repairs of tests that depend on the network were less popular, since they replace a real service with a stub and thus test less of the system. Developers preferred to keep such tests and to mark them for a separate suite that runs once a day.

The cost of the detector lies in the reruns. Ten reruns of a failing test take on average a few seconds for a unit test and a few minutes for an integration test. Since flaky failures are a small part of all builds, the reruns add little to the total time of the pipeline. The detector also caches the outcome of each variation, therefore a test that fails again in a later build needs no new reruns until its code changes.

Several threats limit these results. The labels of the failures come from two authors, who agreed on most cases and resolved the rest in a discussion. The projects use Java and Maven, and other languages and build tools may show other causes of flakiness. Finally, one year of builds may miss causes that appear only after a change of the build machines, such as a new version of the operating system or of the virtual machine.

V. Conclusion

Continuous integration runs the test suite of a project after every change to the code. A failing test stops the merge of the change until a developer finds the cause of the failure and repairs the code or the test. Most failures point to a fault in the change itself. Some failures, however, come from tests that pass and fail on the same code, and these flaky tests cost developers hours of work on faults that do not exist.

Prior studies of open-source projects report that flaky tests account for a large part of the failures that developers investigate. The most common causes are asynchronous waits, concurrency, the order in which tests run, and dependencies on the network or on the clock. Each cause calls for a different repair. A test that waits for a fixed time needs a condition to wait for, while a test that depends on the order of the suite needs its own setup and teardown of the shared state.

The detector of this paper reruns each failing test under controlled variations of its environment. It changes the order of the tests, the timing of the threads, and the seed of every random number generator. A test whose outcome changes under any variation is flaky, and the variation that changes the outcome names the likely cause. The detector reports the test, the variation, and the lines of the test that read the shared state.

We evaluate the detector on the failures of twelve Java projects over one year of builds. The projects range from small libraries with a few hundred tests to large frameworks with tens of thousands of tests. For each failure, the build logs record the commit, the failing tests, and the outcome of the next build on the same commit. A failure that the next build does not repeat is a candidate flaky failure, which two authors then label by hand.

The detector finds the cause of most flaky failures within ten reruns. Order dependence and timing account for the largest groups, followed by network access and random seeds. The remaining failures depend on the state of the build machine, such as a full disk or a busy port, which no variation of the test environment reproduces. These failures call for checks of the machine before the build instead of changes to the test.

Developers accepted most of the repairs that the detector suggested. A repair that adds a condition to a wait or a reset of a shared field takes a few lines of code and leaves the intent of the test intact. Repairs of tests that depend on the network were less popular, since they replace a real service with a stub and thus test less of the system. Developers preferred to keep such tests and to mark them for a separate suite that runs once a day.

The cost of the detector lies in the reruns. Ten reruns of a failing test take on average a few seconds for a unit test and a few minutes for an integration test. Since flaky failures are a small part of all builds, the reruns add little to the total time of the pipeline. The detector also caches the outcome of each variation, therefore a test that fails again in a later build needs no new reruns until its code changes.

Several threats limit these results. The labels of the failures come from two authors, who agreed on most cases and resolved the rest in a discussion. The projects use Java and Maven, and other languages and build tools may show other causes of flakiness. Finally, one year of builds may miss causes that appear only after a change of the build machines, such as a new version of the operating system or of the virtual machine.

REFERENCES

[1] A. Author, “A study of flaky tests,” in *Proc. Conf.*, 2020, pp. 1–10.